# Agent skill 迭代式编写实战

原创 物流技术团队 大淘宝技术 2026年6月12日 00:02 浙江 34人



本文总结Agent Skill编写经验，将其定义为模块化领域知识资产，类似给AI的“操作手册”，适用于半自动化及专家经验导向场景。核心设计遵循三层渐进式披露架构，强调用决策树替代模糊判断、确定性操作脚本化，并建立内部自查与外部评估双重验证机制。Skill本质是以文件系统结构替代复杂运行时服务，实现零依赖部署。相比专用Agent框架，它更轻量但确定性稍弱，旨在将隐性专家经验转化为可复用、可验证的知识资产。

## 01 前言

本文是笔者年初以来摸索 agent skill 编写过程中的经验总结，配图生成自 QoderWork。分享的内容包括了：

- 笔者对于 agent skill 概念和特性的理解，以及适用的场景
- 迭代式编写 agent skill 的实践经验

## 02 关于 skill：概念、特性与应用场景

### 什么是 Agent Skill

Agent Skill 是模块化的能力包，沉淀了领域知识的资产 bundle，资产包括自然语言指令、元数据和可选资源（脚本、模板），让 AI Agent 在需要时自动加载和使用。如果说 MCP 为 agent 提供了"手"来操作工具，那么 skill 就是"操作手册"，教 agent 怎么用这些工具。

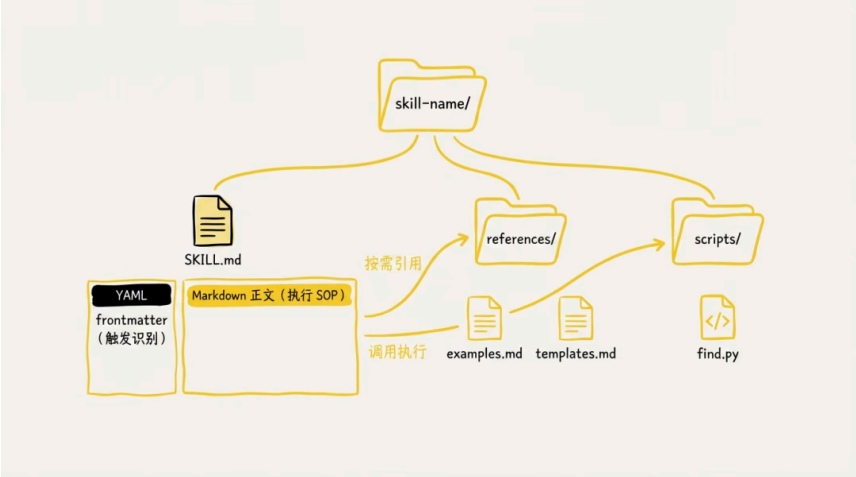
从时间线看，2025 年 10 月中旬 Anthropic 正式发布 Claude Skills，两个月后 Agent Skills 作为开放标准发布，Cursor、OpenCode、Qoder 等主流工具陆续跟进。

通俗理解：skill 就是给 agent 准备的业务 SOP 大礼包，涵盖执行流程、背景知识、工具使用说明、模板素材，以及常见问题的处理方式。

## Agent skill 的形态

skill 基于文件系统驱动，agent 工具通过 skill 标准的 tool 和终端/文件操作能力完成发现与加载。

标准 Skill 目录结构如下：




**SKILL.md** 是整个 skill 的核心，由两部分组成：

- **YAML frontmatter**：skill 的元数据，agent 通过此判断是否触发。

  `description` 是关键，通常写明适用场景和关键触发词。

- **Markdown** 正文：面向 agent 的执行 SOP，建议用「总–分」结构，先说核心规则，再展开约束细节。

**references/** 放补充文档，如模板文件、详细规范、示例代码等，在 SKILL.md 中通过路径按需引用，不要把大量内容堆进主文件。

**scripts/** 放确定性操作的执行脚本（Python/Bash 等）。能写成脚本的就不要靠 agent 推理——脚本的稳定性和可复现性远高于让 agent 猜。

扩展阅读：如何处理带有三方库依赖的 python 脚本：uv run + pep 723

## Agent Skill 的核心特性

| 特性 | 说明 |
| --- | --- |
| 模块化封装 | 专业技能被封装为可组合的能力和资源模块 |
| 渐进式披露 | 技能信息分三层，agent 按需加载，避免上下文窗口过载 |
| 文件系统驱动 | 通过结构化文件夹（指令、脚本、资源）动态加载 |
| 模型无关 | 可跨不同 Agent 运行时复用 |
| 脚本化执行 | 确定性操作交由脚本完成，模型负责决策与编排 |

## 适合使用 Agent Skill 的场景

以下场景特别适合：

- **半自动化重复流程**：同一业务流程频繁执行，部分依赖主观判断，无法完全自动化
- **领域知识导向**：业务流程依赖专家知识，llm 泛化能力难以覆盖
- **上下文受限**：agent 职责多样且上下文窗口有限，处理其他任务时不希望无关知识占位

不适合的场景：

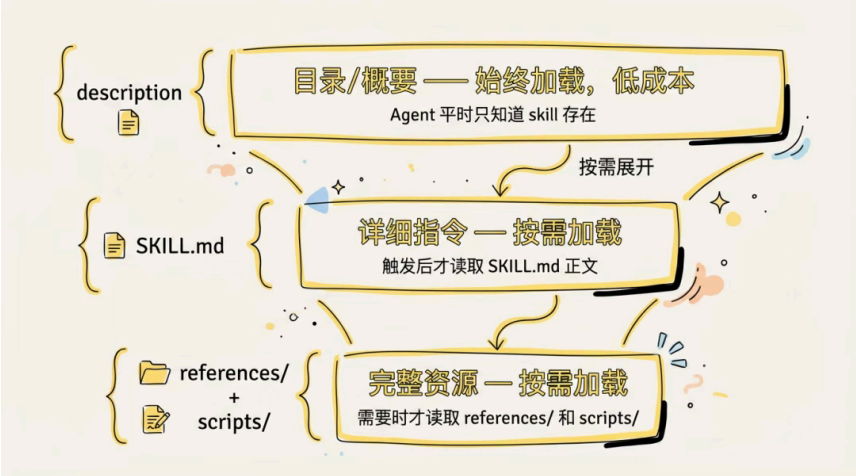
- **简单任务**：直接用基础提示词，靠 llm 泛化能力即可
- **流程完全确定性**：写代码自动化更合适
- **agent 职责高度单一**：把 skill 内容放进 system prompt，脚本用 mcp/tools 代替，没必要包装成 skill

# 03 Agent Skill 编写经验

推荐先装好这两个元 skill 再上手：

- https://skills.sh/anthropics/skills/skill-creator 用于生成 skill
- https://skills.sh/softaworks/agent-toolkit/skill-judge 用于评估 skill

## 设计时遵循三层渐进式加载架构




渐进式披露（Progressive Disclosure）是 Agent Skill 最核心的设计思想，把技能信息拆成三层：

- 第一层：目录/概要（成本最低，agent 平时只需知道手册目录）
- 第二层：详细指令（按需加载，需要时才打开具体章节）
- 第三层：完整资源（包含详细步骤和工具脚本）

设计 skill 时就要想清楚：哪些内容必须在 skill.md，哪些可以下沉到 reference。

| 原则 | 说明 | 示例 |
| --- | --- | --- |
| 技能即知识 | skill 的价值在于领域知识，llm 能解决的别重复说，会分散注意力 | 告诉 agent"查询业务日志 query 构造"，不要告诉 agent 如何读文件、如何开网页 |
| 脚本即能力落点 | 确定性操作交给脚本，模型负责决策与编排 | 计算、查询、结果处理用脚本完成 |
| description 导向触发 | description 的核心是"什么情况下触发"，不只是功能描述 | 除了能做什么，还要说清楚什么时候该用 |
| 精简主文 + 按需引用 | SKILL.md 只写核心指令，详细示例和模板放 references/，通过路径引用 | 问自己"所有场景都需要这段知识吗？"，不是就放到 reference |
| 职责单一 | 一个 skill 只做一件事。复杂工作流拆成多个 skill 组合，各自按需加载，前序任务不会被后续任务的上下文污染 | 代码生成、迁移检查、测试编写分别是三个 skill，而不是一个大而全的 skill |

## 迭代式开发




以下是开发过程中一些具体的可优化 skill 效果的实践

- 用决策树替代模糊判断

决策树是正向约束，能让 agent 在需要做判断时行为可控。以下是一个异步消息问题排查的 skill 的片段示例

```
### 结果处理规则
**补全未发出消息：** 若有序事件的前序有日志、后序无日志，在报告表格中补充后序事件行
**消费失败处理：** 判断某 tag 是否失败，标准为 `resultFlag = N` 且该 tag 后续
- 若后续**有** `Y`（重试成功）→ 取**第一条**失败行，调用错误详情查询
- 若后续**无** `Y`（持续失败）→ 取**每一条**失败行，调用错误详情查询
**错误详情查询（消费失败）：**
```

skill-judge 把决策树列为高质量 skill 的明确标志，在 D1（知识增量）和 D8（实用性）两个维度都是加分项：

> Green flags (indicators of high knowledge delta):- Decision trees for non-obvious choices ("when X fails, try Y because Z")
> D8 Usability 检查项： - Decision trees: For multi-path scenarios, is there clear guidance on which path to take?

原因很简单：skill 的核心价值是封装专家才有的判断知识。决策树把"应该怎么判断"写清楚，而不是用模糊语言把判断压力甩给 agent。agent 不需要推理，顺着树走就行。比模糊描述的可执行性高很多。写 skill 时遇到分支判断，优先用树形结构，而不是让 agent 自行决策。

- 负向约束要配替代方案

告诉 agent 不能做什么（bad case/anti pattern）时，同步给出合法替代方案，约束力会强很多。以下是一个单元测试编写 skill 的片段，给出了 mock 时的 bad case 和相应的解决方案

```
### Mocking Restrictions
**Do NOT mock:**
- `public static` fields (e.g., `@AppSwitch`-annotated configurations
  restore originals post-test
- POJO classes or OneLog objects - initialize simple POJOs programmat
- Stateless static methods (e.g., utility methods for conversion/asser
```

不要只说"不能做什么"，要给"那应该怎么做"。这也是一种 few shot。没有替代方案的话，agent 会自己找一个——结果往往不是你想要的。

- skill 执行后自查机制

决策树解决的是执行前的分支判断——走哪条路。自查机制解决的是另一半：执行完了，产出物是否合格。以下是一个单元测试生成 skill 的执行后自查列表示例

```
## Post-Generation Review
After generating tests, review against this specification to ensure:
- Correct test file location and naming
- Proper mock configuration without prohibited patterns
- Complete verification of return values, state mutations, and invoca
- AssertJ assertion patterns are used consistently
- No reflection-based testing or private member verification
- Similar tests are grouped into parameterized tests where appropriat
- Parameterized tests use appropriate source types and handle null va
```

agent 的自然倾向是完成任务就结束，不会主动回头检验。很多规范性错误恰恰在"完成"之后才能发现。把自查写进 skill，就是强制插入一个反射节点，把"我觉得做完了"变成"我验证过做完了"。

自查可以从两个角度来：

- 规范符合性：对照约束检查，确认没做错
- 覆盖完整性：对照领域知识检查，确认没遗漏

决策树是收敛的（从多条路中选一条），自查清单是发散的（从一个结果出发，在多个维度验证）。有明确输出规范的 skill（代码生成、迁移、测试等），skill 成熟后建议补上自查机制。

- skill 的外部验证：eval 机制

自查是 skill 执行时的内部静态验证，由 agent 对照清单自检。skill-creator 还提供了外部动态验证：用真实输入跑 skill，对比有无 skill 的输出差距，而不是靠感觉判断好坏。

eval 的四个环节：

- **测试用例**：设计 2-3 个真实用户会说的提示词，同一提示词分别跑「有 skill」和「无 skill（或旧版本）」，保留两份输出用于对比。提示词要有足够的复杂度和具体背景——skill 的触发本身依赖 agent 对 description 的语义识别，过于简单的 prompt 可能根本不会触发 skill，也就测不出任何东西
- **断言**：有客观标准的 skill 设计可验证检查项（如"输出文件是否包含字段 X"），主观类 skill 基于人工反馈
- **迭代循环**：评估 → 修改 → 重跑 → 再评估，每轮聚焦有明确问题的用例，直到没有明显差距。注意：每轮只看少数用例，容易把 skill 改成只对这几个 case 有效。改的时候要从具体反馈里抽出通用规律，而不是针对测试用例做针对性修补
- **description 触发率优化**：skill 内容稳定后单独优化 description，用 should-trigger / should-not-trigger 样本测试召回精度，重点关注"近似场景误触发"和"该触发却未触发"两类边界

内部自查是运行时护栏，外部 eval 是开发期的标准线，定位不同，都有用。

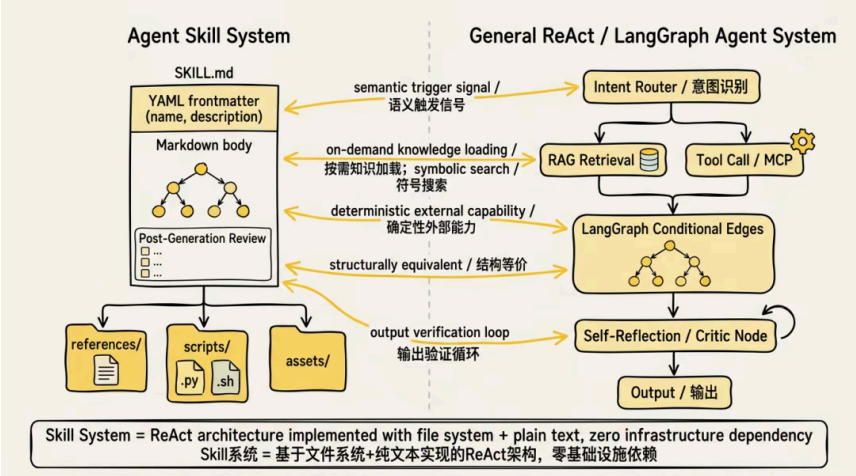
## 多人协作 skill 管理

skills.sh 提供了配套的 skill 管理工具。如果要多人协作共享 skill，可以在 code 平台上建一个仓库，根目录放 skills 目录，下面存放各个 skill。需要用时运行以下命令交互式安装：




## 扩展视角：skill 是简化版的专用 agent

熟悉 ReAct / LangGraph 等框架的话，可以用这张映射表快速建立认知：

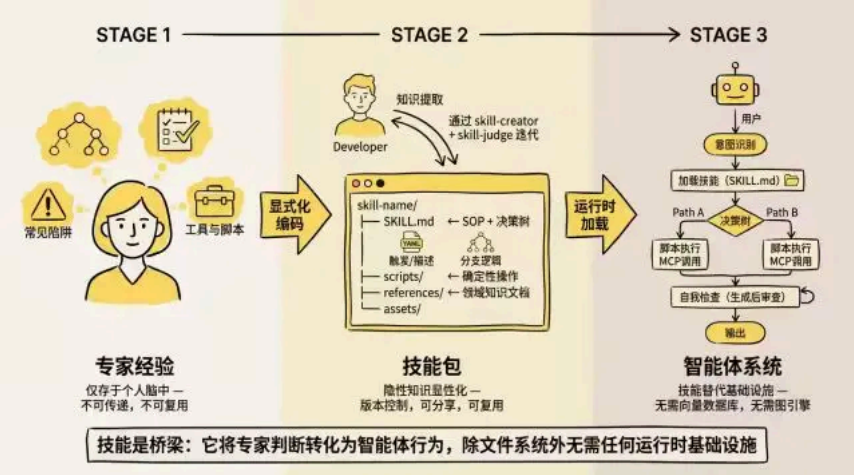



| Skill 概念 | 通用 Agent 体系对应 | 说明 |
| --- | --- | --- |
| YAML `description` | Intent Router / 意图识别 | 触发识别的 semantic signal，决定调用哪个 skill |
| `references/` + bash find/grep | RAG 检索层/外部知识库 | 按需加载的外部知识；bash 是符号检索，非向量语义检索 |
| `scripts/` | MCP Tool / Function Call | 本质相同：模型可调用的外部确定性能力 |
| Decision Tree | LangGraph 节点 + 条件边 | 结构等价，但实现层不同（见下方） |
| Post-Generation Review | Critic / Self-Reflection Node | ReAct 的 Observation 步骤 |
| 渐进式披露（三层） | Dynamic Context Management | 动态扩展 context window 的策略 |

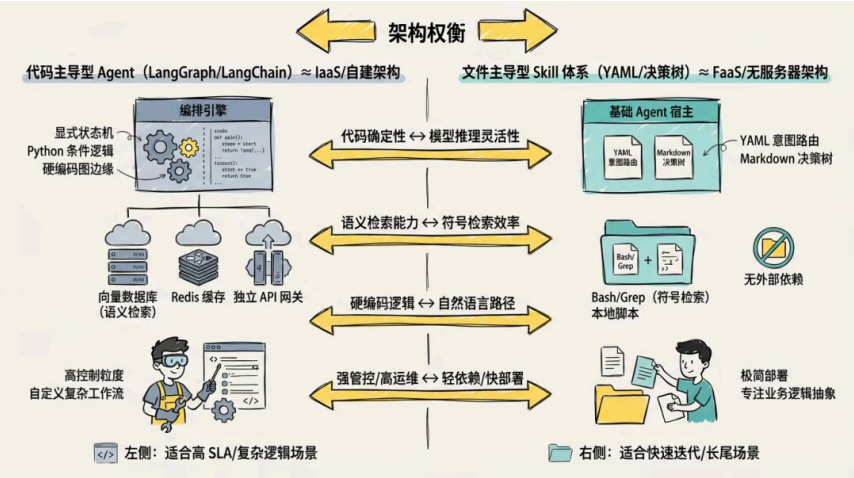
LangGraph 的条件边由调度器执行，skill 里的决策树由模型阅读并模拟——前者是代码控制流，后者是自然语言编码的推理路径。这个差异决定了二者的适用边界。

Skill 体系的本质：用文件系统结构 + 文本决策树，替代运行时服务（向量库、图引擎、路由服务），以零基础设施依赖换取极简部署。确定性不如专用 agent，但对大多数专业流程

场景够用。




以 web 概念类比，基于 langChain 等框架开发相当于 IaaS/SaaS——自建 workflow、管理上下文、实现 tools、接入 mcp。skill 更像 FaaS，agent 工具作为基座已经提供了 skill 的发现与加载，以及基于 bash 的文本操作、脚本执行能力，skill 专注业务流程抽象就够了。需要更高准确性/SLA 时，也可以选择把 skill "翻译"为专用 agent：用 mcp/tools 替代脚本，用流程编排替代决策树，用 observation 节点替代自查。




FaaS 不适合长连接、高频低延迟或复杂有状态事务。同样，对流程确定性要求 100%、或逻辑复杂到 context 爆炸的场景，skill 也撑不住，还是要回到 LangGraph 定制专用 agent。

## 04 总结

本文系统梳理了 Agent Skill 从概念理解到迭代编写的完整实践路径。核心要点可归纳为三点：

- 一是明确定位，Skill 本质是轻量级的领域知识封装，适用于半自动化、专家经验导向的场景，而非替代专用 Agent 框架；
- 二是遵循设计原则，采用三层渐进式加载架构，用决策树替代模糊判断，负向约束配合替代方案，并建立内部自查与外部 eval 的双重验证机制；
- 三是迭代式开发，借助 skill-creator 和 skill-judge 等工具，通过"生成→评估→修订"的快速循环提升 Skill 质量。

对于希望将业务经验沉淀为可复用 AI 能力的团队，Skill 提供了一条零基础设施依赖、低部署成本的可行路径。但需谨记其适用边界——当流程确定性要求 100% 或逻辑复杂度导致上下文爆炸时，仍可选择 LangGraph 等专用 Agent 框架进行定向优化。最终，Skill 的价值不在于技术本身，而在于能否将隐性专家经验转化为显性、可执行、可验证的知识资产。




本文作者其林，来自淘天集团-物流技术团队。本团队深耕于物流的数字化协同与运营领域；为淘天业务提供多样的物流经营管理方案及工具；为商家提供高效低成本的物流解决方案；为消费者提供便捷靠谱的物流体验。我们团队有自研的先进的技术框架，有成熟的敏捷实践开发模式，以先进的技术框架及开发模式不断升级自己，以满足和驱动庞大、复杂、多变的电商物流体系发展。

**拓展阅读**

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 9690

## 留言 2

写留言



桥口 山东 6月13日

点赞，再有详细的实际案例就好了



墨香同学的... 浙江 6月12日

🤭